

HARARE INSTITUTE OF TECHNOLOGY

INFORMATION, SCIENCE AND TECHNOLOGY

SOFTWARE ENGINEERING

OOP ASSIGNMENT

MUKONORI TAPIWA D.

H250215Z

Lecturer: Mr Chiworera

OCTOBER 7

BANK DEMO CLASS

```
import java.util.ArrayList;
import java.util.List;

public class BankDemo {
    public static void main(String[] args) {
        List<Account> accounts = new ArrayList();
        accounts.add(new SavingsAccount("SAV-001", (double)1000.0F,
(double)100.0F));
        accounts.add(new CurrentAccount("CUR-001", (double)200.0F,
(double)300.0F));
        accounts.add(new SavingsAccount("SAV-002", (double)150.0F,
(double)100.0F));
        System.out.println("=== Round 1: withdraw 100 from every account ===");

        for(Account a : accounts) {
            a.withdraw((double)100.0F);
        }

        System.out.println("\n=== Round 2: withdraw 350 from every account
===");

        for(Account a : accounts) {
            a.withdraw((double)350.0F);
        }

        System.out.println("\n=== Round 3: withdraw 100 again (tests the
overdraft limit) ===");

        for(Account a : accounts) {
            a.withdraw((double)100.0F);
        }

        System.out.println("\n=== Month end ===");

        for(Account a : accounts) {
            a.endOfMonth();
        }

        System.out.println("\n=== Final balances ===");

        for(Account a : accounts) {
            System.out.printf("%s -> %.2f%n", a.accountNumber, a.getBalance());
        }

    }
}
```

ACCOUNT CLASS

```
public abstract class Account {
    protected String accountNumber;
    protected double balance;

    public Account(String accountNumber, double openingBalance) {
        this.accountNumber = accountNumber;
        this.balance = openingBalance;
    }

    public void deposit(double amount) {
        if (amount <= (double)0.0F) {
            System.out.println("[" + this.accountNumber + "] Deposit
rejected: amount must be positive.");
        } else {
            this.balance += amount;
            System.out.printf("[%s] Deposited %.2f. Balance: %.2f%n",
this.accountNumber, amount, this.balance);
        }
    }

    public double getBalance() {
        return this.balance;
    }

    public abstract void withdraw(double var1);

    public abstract void endOfMonth();
}
```

CURRENTACCOUNT CLASS

```
public class CurrentAccount extends Account {
    private static final double MONTHLY_FEE = (double)5.0F;
    private final double overdraftLimit;

    public CurrentAccount(String accountNumber, double openingBalance,
double overdraftLimit) {
        super(accountNumber, openingBalance);
        this.overdraftLimit = overdraftLimit;
    }

    public void withdraw(double amount) {
        if (amount <= (double)0.0F) {
            System.out.println("[" + this.accountNumber + "]
Withdrawal rejected: amount must be positive.");
        } else if (this.balance - amount < -this.overdraftLimit) {
            System.out.printf("[%s] CURRENT withdrawal of %.2f
REJECTED: would exceed overdraft limit of %.2f.%n",
this.accountNumber, amount, this.overdraftLimit);
        } else {
            this.balance -= amount;
            String note = this.balance < (double)0.0F ? " (in
overdraft)" : "";
            System.out.printf("[%s] CURRENT withdrew %.2f.
Balance: %.2f%s%n", this.accountNumber, amount, this.balance, note);
        }

    }

    public void endOfMonth() {
        this.balance -= (double)5.0F;
        System.out.printf("[%s] CURRENT month-end: fee %.2f deducted.
Balance: %.2f%n", this.accountNumber, (double)5.0F, this.balance);
    }
}
```

SAVINGSACCOUNT CLASS

```
public class SavingsAccount extends Account {
    private static final double INTEREST_RATE = 0.02;
    private final double minimumBalance;

    public SavingsAccount(String accountNumber, double openingBalance,
double minimumBalance) {
        super(accountNumber, openingBalance);
        this.minimumBalance = minimumBalance;
    }

    public void withdraw(double amount) {
        if (amount <= (double)0.0F) {
            System.out.println("[" + this.accountNumber + "]
Withdrawal rejected: amount must be positive.");
        } else if (this.balance - amount < this.minimumBalance) {
            System.out.printf("[%s] SAVINGS withdrawal of %.2f
REJECTED: balance would fall below the minimum of %.2f.%n",
this.accountNumber, amount, this.minimumBalance);
        } else {
            this.balance -= amount;
            System.out.printf("[%s] SAVINGS withdrew %.2f.
Balance: %.2f%n", this.accountNumber, amount, this.balance);
        }

    }

    public void endOfMonth() {
        double interest = this.balance * 0.02;
        this.balance += interest;
        System.out.printf("[%s] SAVINGS month-end: interest %.2f
added. Balance: %.2f%n", this.accountNumber, interest, this.balance);
    }
}
```

SAMPLE OUTPUT

```
=== Round 1: withdraw 100 from every account ===
[SAV-001] SAVINGS withdrew 100.00. Balance: 900.00
[CUR-001] CURRENT withdrew 100.00. Balance: 100.00
[SAV-002] SAVINGS withdrawal of 100.00 REJECTED: balance would fall below the minimum of 100.00.

=== Round 2: withdraw 350 from every account ===
[SAV-001] SAVINGS withdrew 350.00. Balance: 550.00
[CUR-001] CURRENT withdrew 350.00. Balance: -250.00 (in overdraft)
[SAV-002] SAVINGS withdrawal of 350.00 REJECTED: balance would fall below the minimum of 100.00.

=== Round 3: withdraw 100 again (tests the overdraft limit) ===
[SAV-001] SAVINGS withdrew 100.00. Balance: 450.00
[CUR-001] CURRENT withdrawal of 100.00 REJECTED: would exceed overdraft limit of 300.00.
[SAV-002] SAVINGS withdrawal of 100.00 REJECTED: balance would fall below the minimum of 100.00.

=== Month end ===
[SAV-001] SAVINGS month-end: interest 9.00 added. Balance: 459.00
[CUR-001] CURRENT month-end: fee 5.00 deducted. Balance: -255.00
[SAV-002] SAVINGS month-end: interest 3.00 added. Balance: 153.00

=== Final balances ===
SAV-001 -> 459.00
CUR-001 -> -255.00
SAV-002 -> 153.00
```